

第九章 虚拟内存：系统

教 师：邹翔宇

计算机科学与技术学院

哈尔滨工业大学（深圳）

主要内容

- 一个小内存系统示例
- 案例研究: Core i7/Linux 内存系统
- 内存映射

Review of Symbols 符号回顾

- 基本参数

- $N = 2^n$: 虚拟地址空间中的地址数量
- $M = 2^m$: 物理地址空间中的地址数量
- $P = 2^p$: 页的大小 (bytes)

- 虚拟地址组成部分

- TLBI: TLB 索引
- TLBT: TLB 标记
- VPO: 虚拟页面偏移量 (字节)
- VPN: 虚拟页号

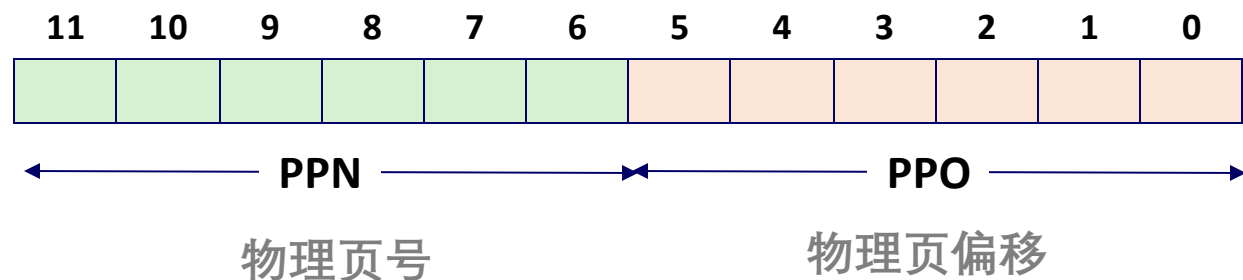
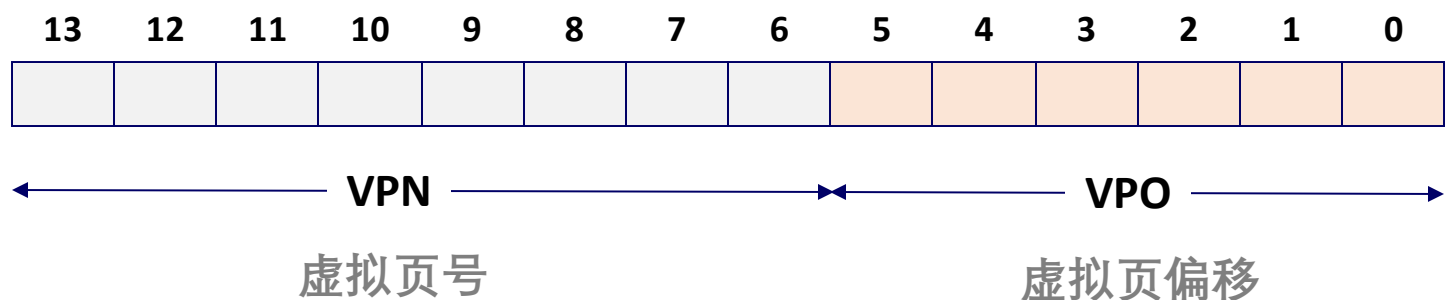
- 物理地址组成部分

- PPO: 物理页面偏移量 (same as VPO)
- PPN: 物理页号
- CO: 缓冲块内的字节偏移量
- CI: Cache 索引
- CT: Cache 标记

一个小内存系统示例

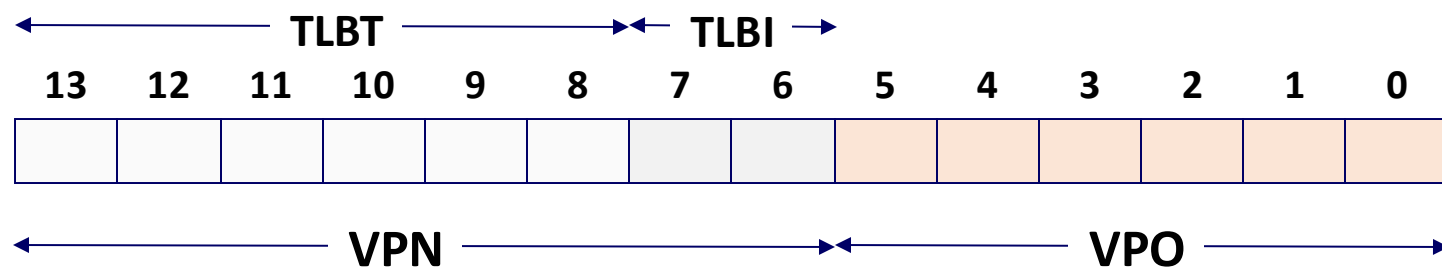
- 地址假设

- 14位虚拟地址 ($n=14$)
- 12位物理地址 ($m=12$)
- 页面大小64字节 ($P=64$)



1. 小内存系统的 TLB

- 16 entries 16个条目
- 4-way associative 4路组相联



组	标记	PPN	有效位	标记	PPN	有效位	标记	PPN	有效位	标记	PPN	有效位
0	03	—	0	09	0D	1	00	—	0	07	02	1
1	03	2D	1	02	—	0	04	—	0	0A	—	0
2	02	—	0	08	—	0	06	—	0	03	—	0
3	07	—	0	03	0D	1	0A	34	1	02	—	0

2. 小内存系统的页表

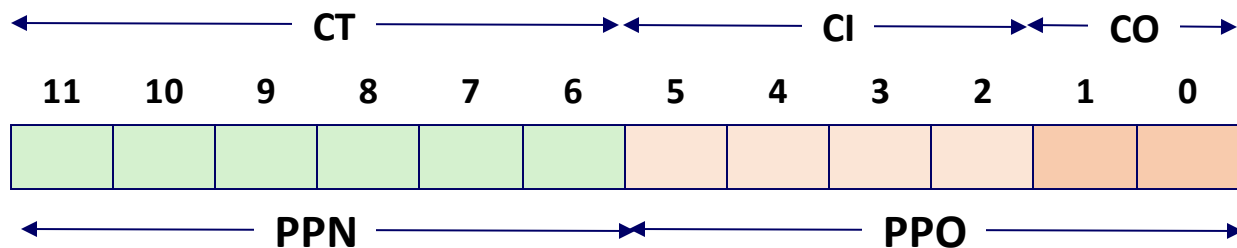
只展示了前16个PTE (out of 256)

<i>VPN</i>	<i>PPN</i>	<i>有效位</i>
00	28	1
01	—	0
02	33	1
03	02	1
04	—	0
05	16	1
06	—	0
07	—	0

<i>VPN</i>	<i>PPN</i>	<i>有效位</i>
08	13	1
09	17	1
0A	09	1
0B	—	0
0C	—	0
0D	2D	1
0E	11	1
0F	0D	1

3. 小内存系统的 Cache

- 16个组，每块为4字节
- 通过物理地址中的字段寻址
- 直接映射

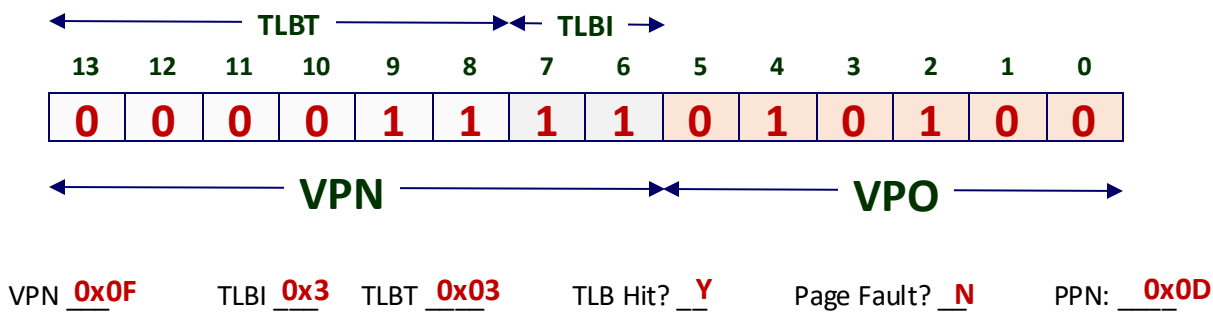


索引	标记位	有效位	块0	块1	块2	块3
0	19	1	99	11	23	11
1	15	0	—	—	—	—
2	1B	1	00	02	04	08
3	36	0	—	—	—	—
4	32	1	43	6D	8F	09
5	0D	1	36	72	F0	1D
6	31	0	—	—	—	—
7	16	1	11	C2	DF	03

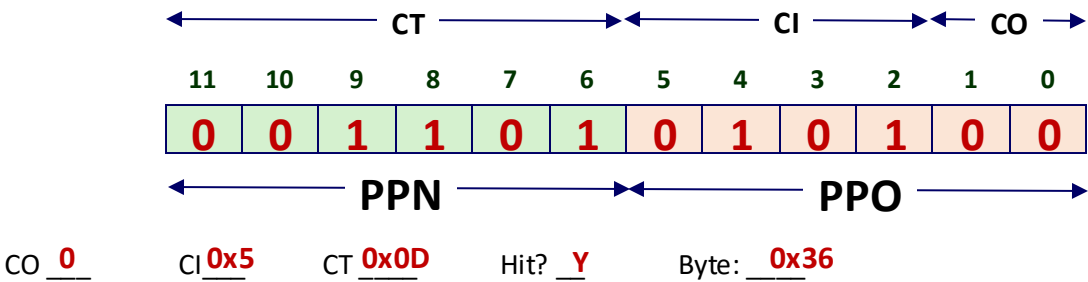
Idx	Tag	Valid	B0	B1	B2	B3
8	24	1	3A	00	51	89
9	2D	0	—	—	—	—
A	2D	1	93	15	DA	3B
B	0B	0	—	—	—	—
C	12	0	—	—	—	—
D	16	1	04	96	34	15
E	13	1	83	77	1B	D3
F	14	0	—	—	—	—

地址翻译 Example #1

虚拟地址: 0x03D4



物理地址



TLB

组	标记	PPN	有效位	标记	PPN	有效位	标记	PPN	有效位	标记	PPN	有效位
0	03	-	0	09	0D	1	00	-	0	07	02	1
1	03	2D	1	02	-	0	04	-	0	0A	-	0
2	02	-	0	08	-	0	06	-	0	03	-	0
3	07	-	0	03	0D	1	0A	34	1	02	-	0

页表

VPN	PPN	有效位
00	28	1
01	-	0
02	33	1
03	02	1
04	-	0
05	16	1
06	-	0
07	-	0

VPN	PPN	有效位
08	13	1
09	17	1
0A	09	1
0B	-	0
0C	-	0
0D	2D	1
0E	11	1
0F	0D	1

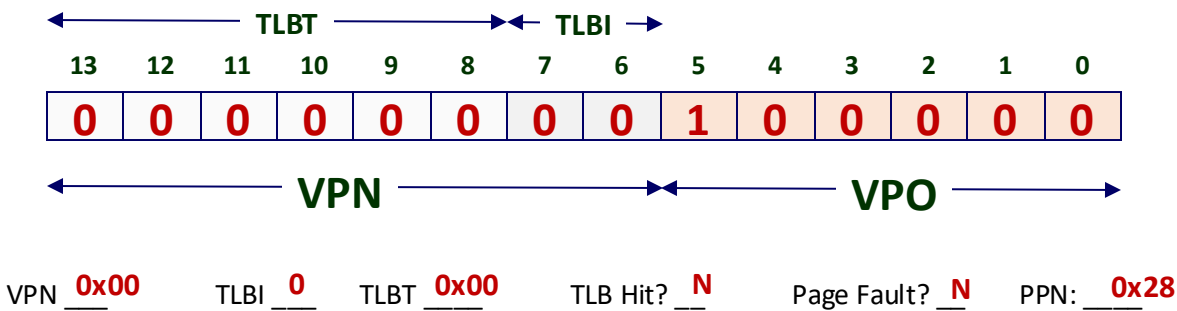
Cache

索引	标记位	有效位	块0	块1	块2	块3
0	19	1	99	11	23	11
1	15	0	-	-	-	-
2	18	1	00	02	04	08
3	36	0	-	-	-	-
4	32	1	43	6D	8F	09
5	0D	1	36	72	F0	1D
6	31	0	-	-	-	-
7	16	1	11	C2	DF	03

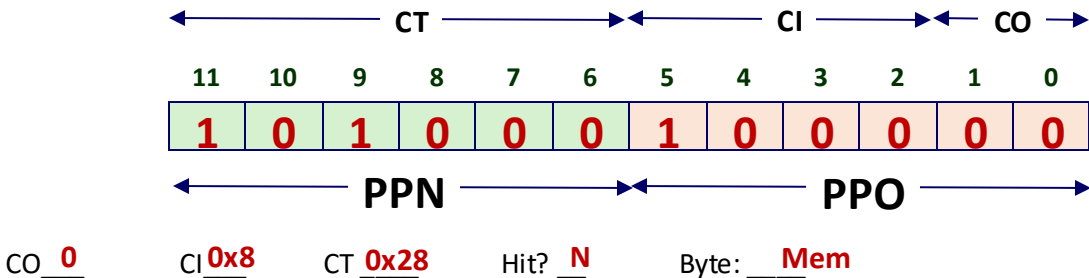
Idx	Tag	Valid	B0	B1	B2	B3
8	24	1	3A	00	51	89
9	2D	0	-	-	-	-
A	2D	1	93	15	DA	3B
B	0B	0	-	-	-	-
C	12	0	-	-	-	-
D	16	1	04	96	34	15
E	13	1	83	77	1B	D3
F	14	0	-	-	-	-

地址翻译 Example #2

虚拟地址: 0x0020



物理地址



TLB

组	标记	PPN	有效位	标记	PPN	有效位	标记	PPN	有效位	标记	PPN	有效位
0	03	-	0	09	0D	1	00	-	0	07	02	1
1	03	2D	1	02	-	0	04	-	0	0A	-	0
2	02	-	0	08	-	0	06	-	0	03	-	0
3	07	-	0	03	0D	1	0A	34	1	02	-	0

页表

VPN	PPN	有效位
00	28	1
01	-	0
02	33	1
03	02	1
04	-	0
05	16	1
06	-	0
07	-	0

VPN	PPN	有效位
08	13	1
09	17	1
0A	09	1
0B	-	0
0C	-	0
0D	2D	1
0E	11	1
0F	0D	1

Cache

索引	标记位	有效位	块0	块1	块2	块3
0	19	1	99	11	23	11
1	15	0	-	-	-	-
2	18	1	00	02	04	08
3	36	0	-	-	-	-
4	32	1	43	6D	8F	09
5	0D	1	36	72	F0	1D
6	31	0	-	-	-	-
7	16	1	11	C2	DF	03

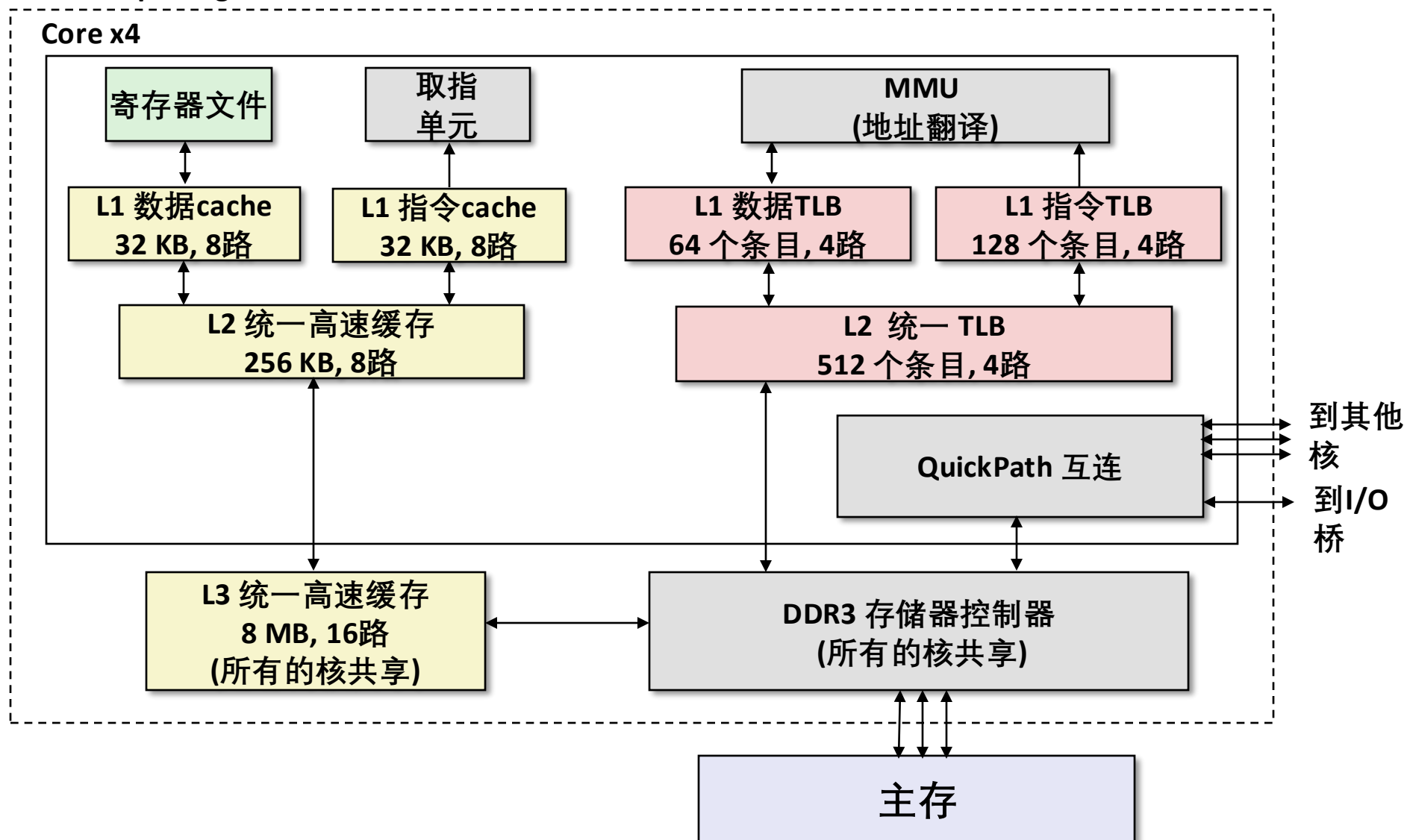
Idx	Tag	Valid	B0	B1	B2	B3
8	24	1	3A	00	51	89
9	2D	0	-	-	-	-
A	2D	1	93	15	DA	3B
B	0B	0	-	-	-	-
C	12	0	-	-	-	-
D	16	1	04	96	34	15
E	13	1	83	77	1B	D3
F	14	0	-	-	-	-

主要内容

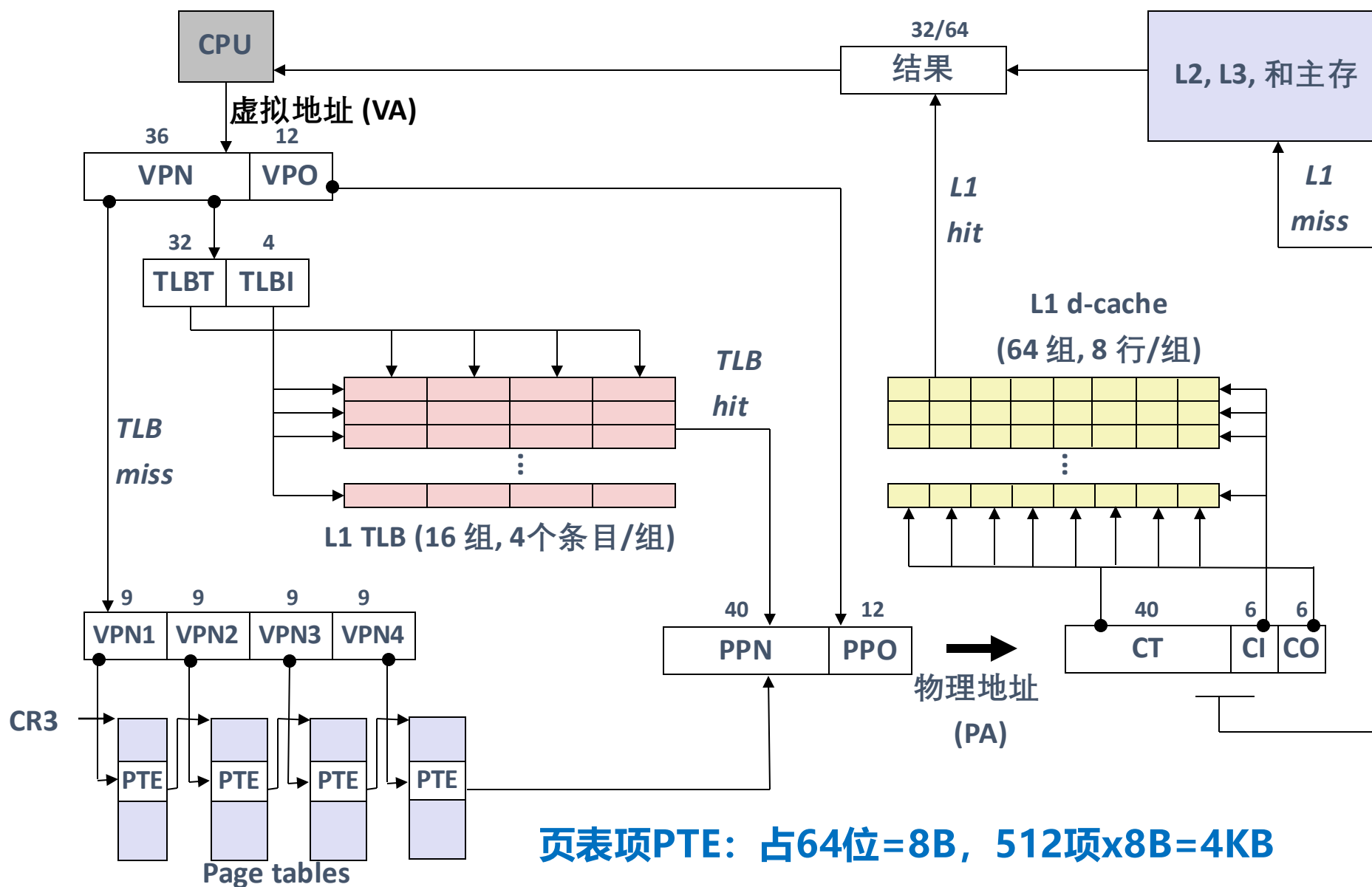
- 一个小内存系统示例
- 案例研究: Core i7/Linux 内存系统
- 内存映射

Intel Core i7 内存系统

Processor package



Core i7 地址翻译 (VA48位PA52位)



Core i7 1-3级页表条目格式

63	62	52	51	12	11	9	8	7	6	5	4	3	2	1	0
XD	未使用	页表物理基地址				未使用	G	PS		A	CD	WT	U/S	R/W	P=1
OS可用 (磁盘上的页表位置)															P=0

每个条目引用一个 4KB子页表:

P: 子页表在物理内存中 (1)不在 (0).

R/W: 对于所有可访问页，只读或者读写访问权限.

U/S: 对于所有可访问页，用户或超级用户 (内核)模式访问权限.

WT: 子页表的直写或写回缓存策略.

A: 引用位 (由MMU 在读或写时设置，由软件清除).

PS: 页大小为4 KB 或 4 MB (只对第一层PTE定义).

Page table physical base address: 子页表的物理基地址的最高40位 (强制页表4KB 对齐)

XD: 能/不能从这个PTE可访问的所有页中取指令.

对照书
p578

Core i7 第 4 级页表条目格式

63	62	52	51	12	11	9	8	7	6	5	4	3	2	1	0
XD	未使用	物理页号				未使用	G		D	A	CD	WT	U/S	R/W	P=1
OS可用 (磁盘上的页表位置)															P=0

每个条目引用一个 4KB子页表:

P: 子页表在物理内存中 (1)不在 (0).

R/W: 对于所有可访问页，只读或者读写访问权限.

U/S: 对于所有可访问页，用户或超级用户 (内核)模式访问权限.

WT: 子页表的直写或写回缓存策略.

A:引用位 (由MMU 在读或写时设置，由软件清除).

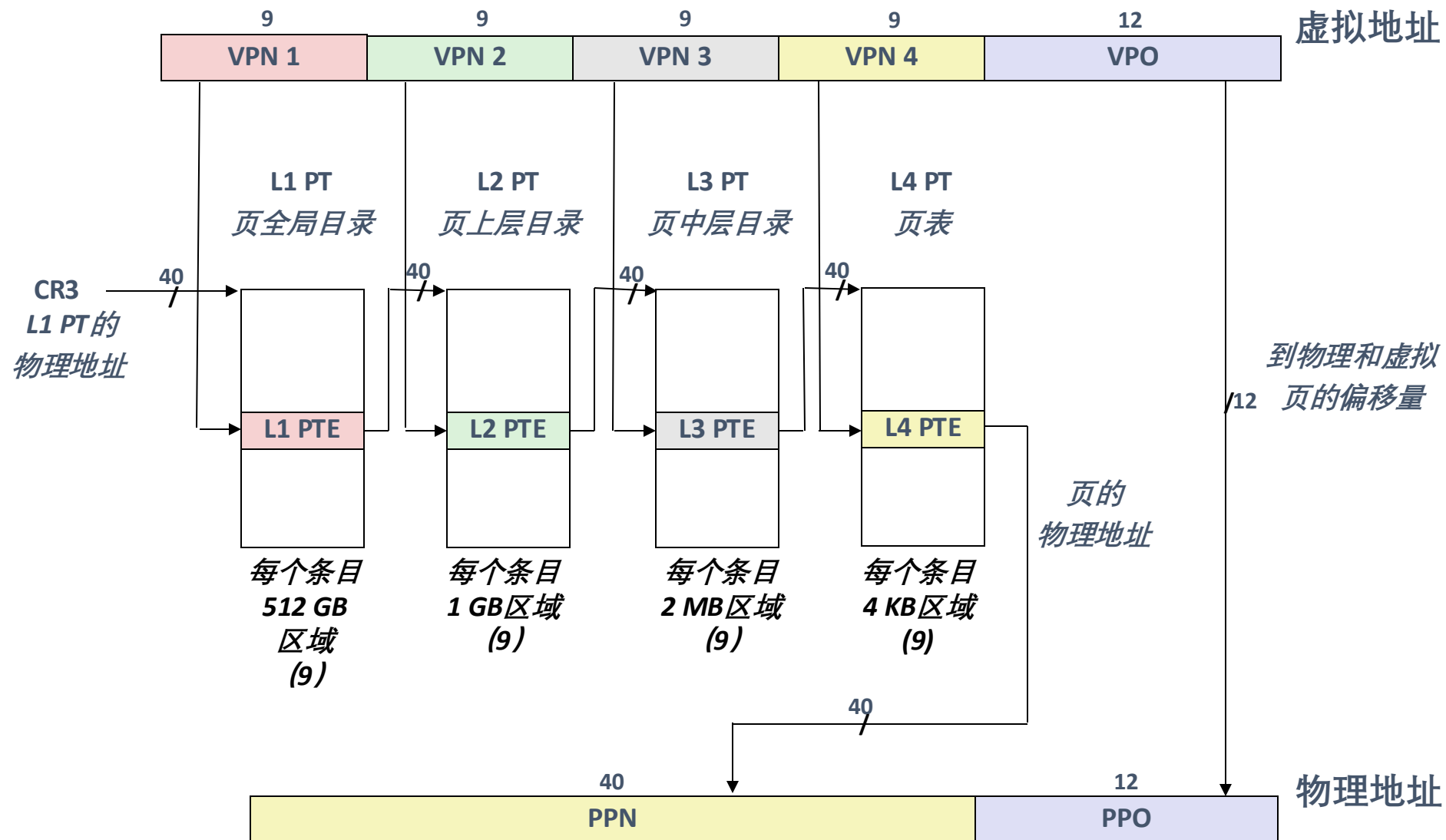
D: 修改位 (由MMU 在读和写时设置，由软件清除)

Page table physical base address: 子页表的物理基地址的最高40位 (强制页表 4KB 对齐)

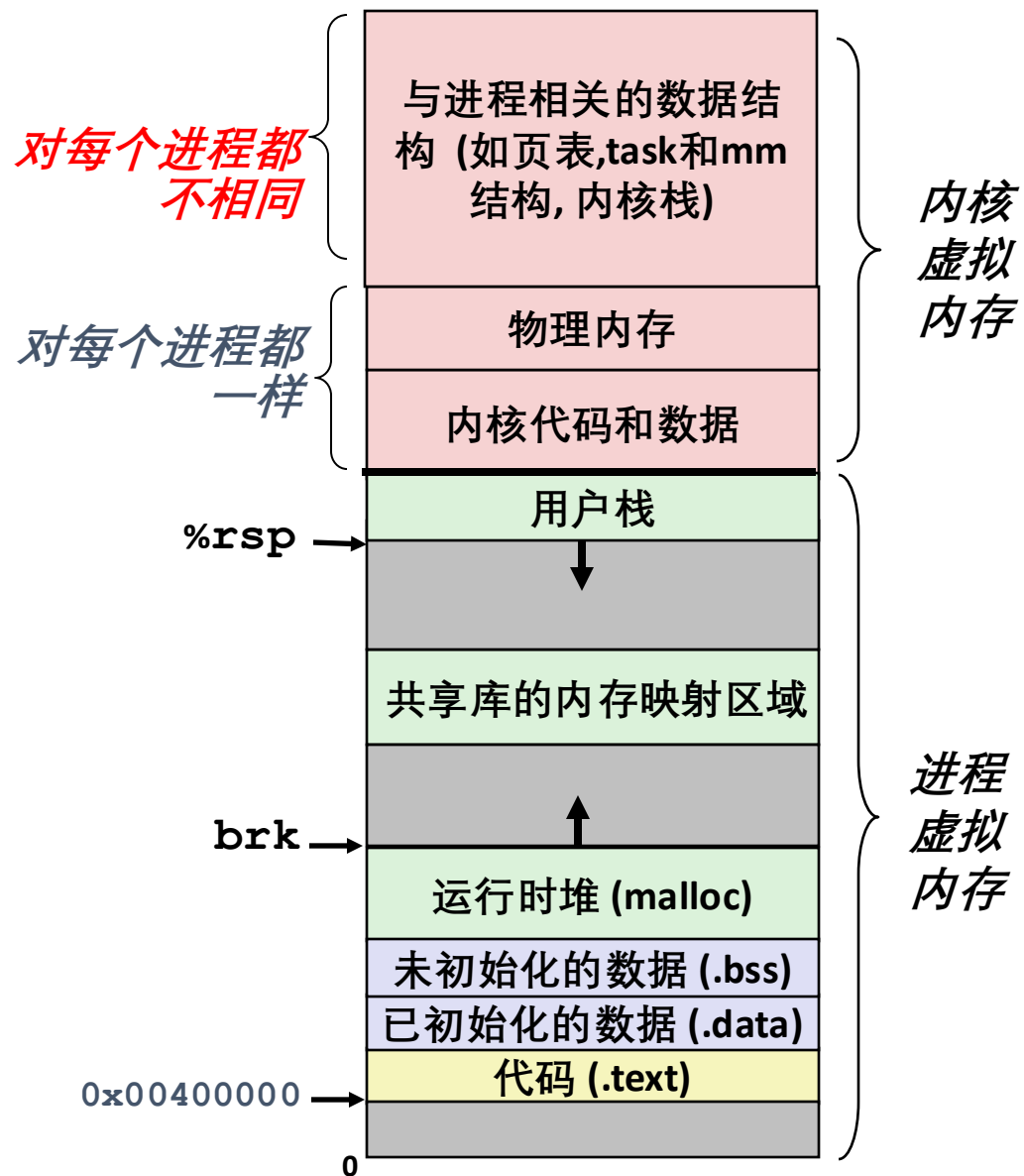
XD: 能/不能从这个PTE可访问的所有页中取指令.

对照书
p578

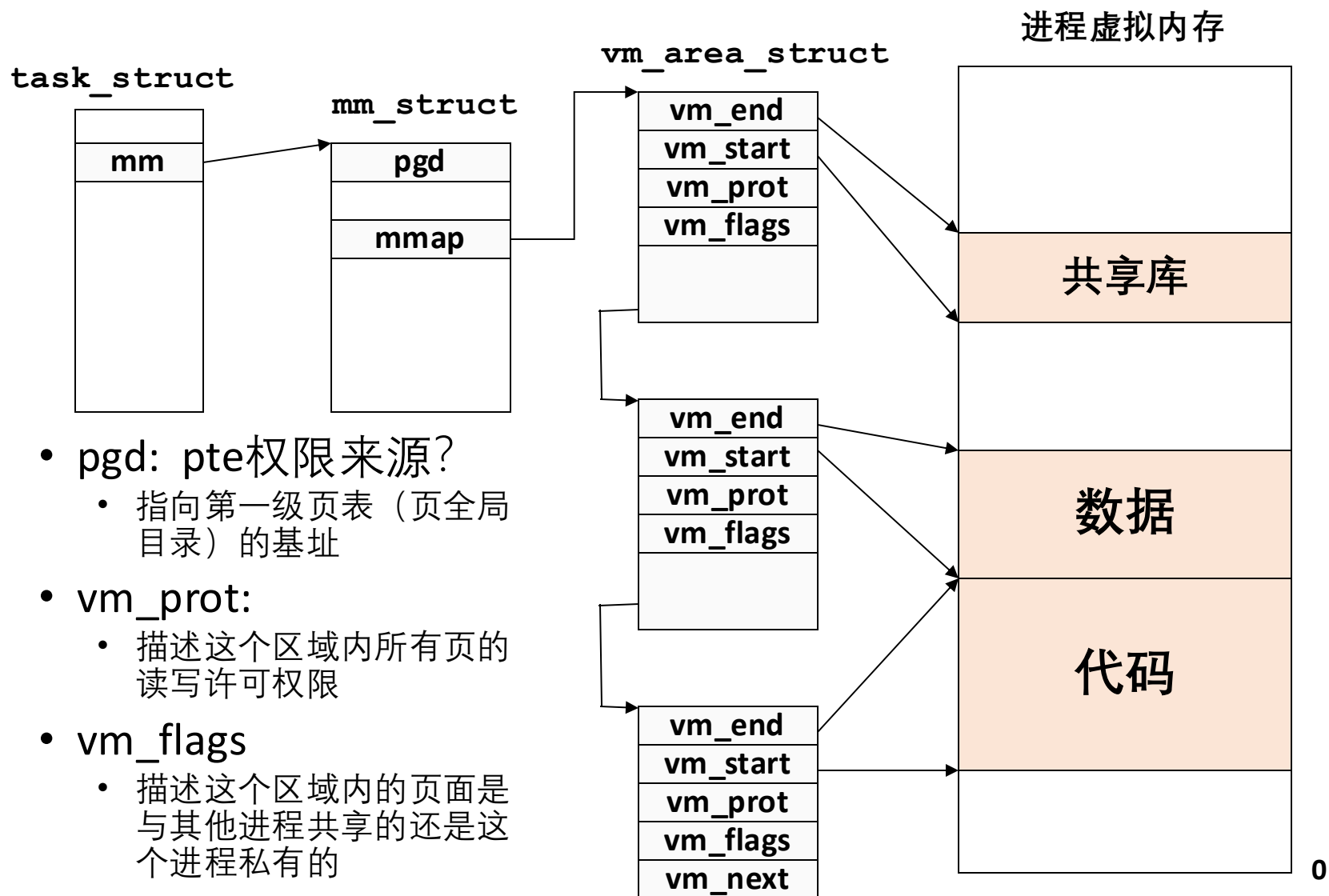
Core i7 页表翻译



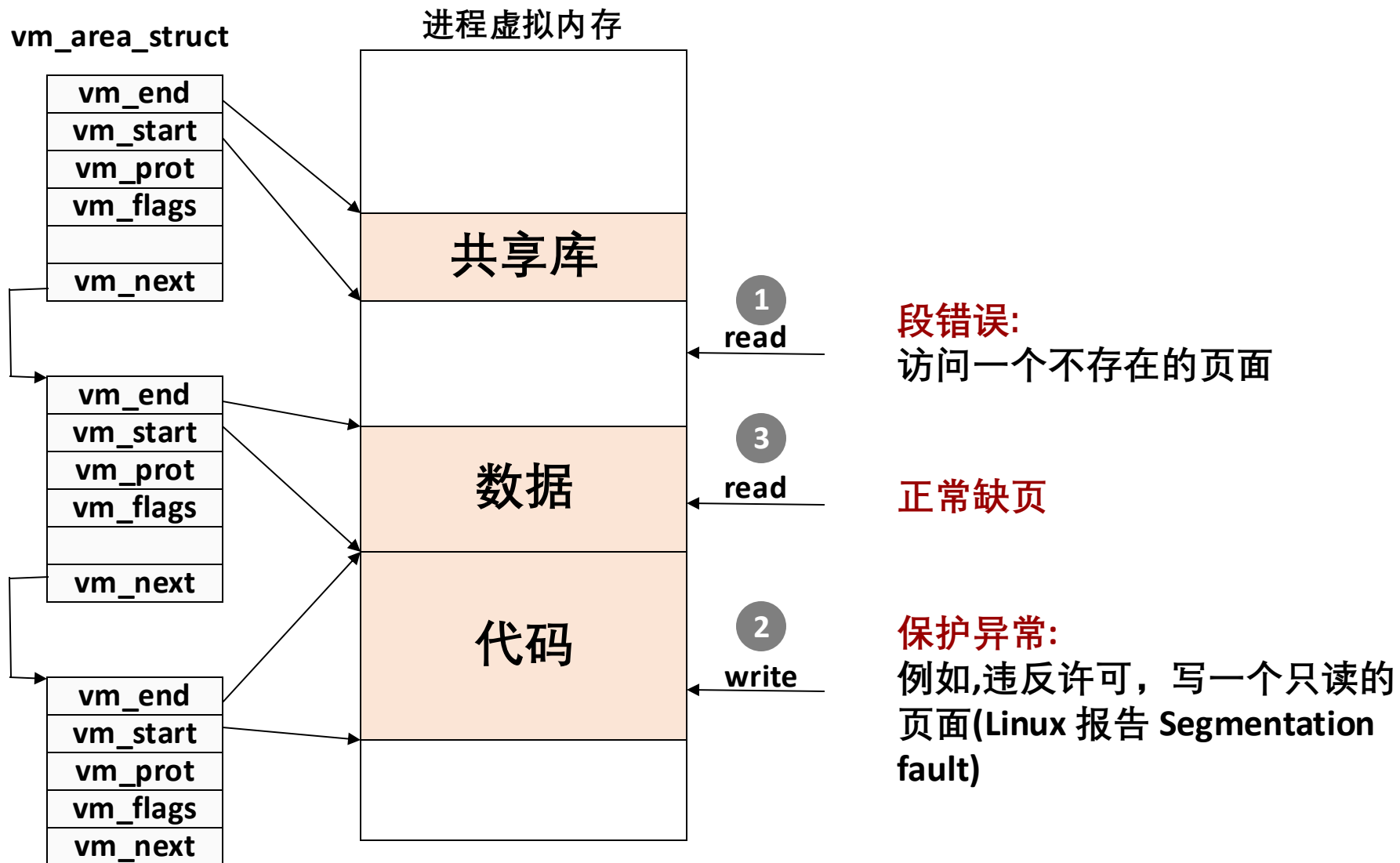
一个Linux进程的虚拟地址空间



Linux将虚拟内存组织成一些区域的集合



Linux缺页处理



主要内容

- 一个小内存系统示例
- 案例研究: Core i7/Linux 内存系统
- 内存映射

内存映射

- Linux通过将虚拟内存区域与磁盘上的对象关联起来以初始化这个虚拟内存区域的内容。
 - 这个过程称为内存映射 (*memory mapping*) .
- 虚拟内存区域可以映射的对象 (根据初始值的不同来源分):
 - 普通映射: 磁盘上的 **普通文件** (e.g., 一个可执行目标文件)
 - 文件区被分成页大小的片, 对虚拟页面初始化
 - **匿名映射: 匿名页、匿名文件** (内核创建, 全是二进制零)
 - 第一次引用该区域内的虚拟页面时分配一个全是零的物理页 (*demand-zero page 请求二进制零的页*)
 - 一旦该页面被修改, 即和其他页面一样
- 初始化后的页面在内存和交换文件 (*swap file*) 之间换来换去

请查看https://blog.csdn.net/sinat_41619762/article/details/118972509

用户级内存映射

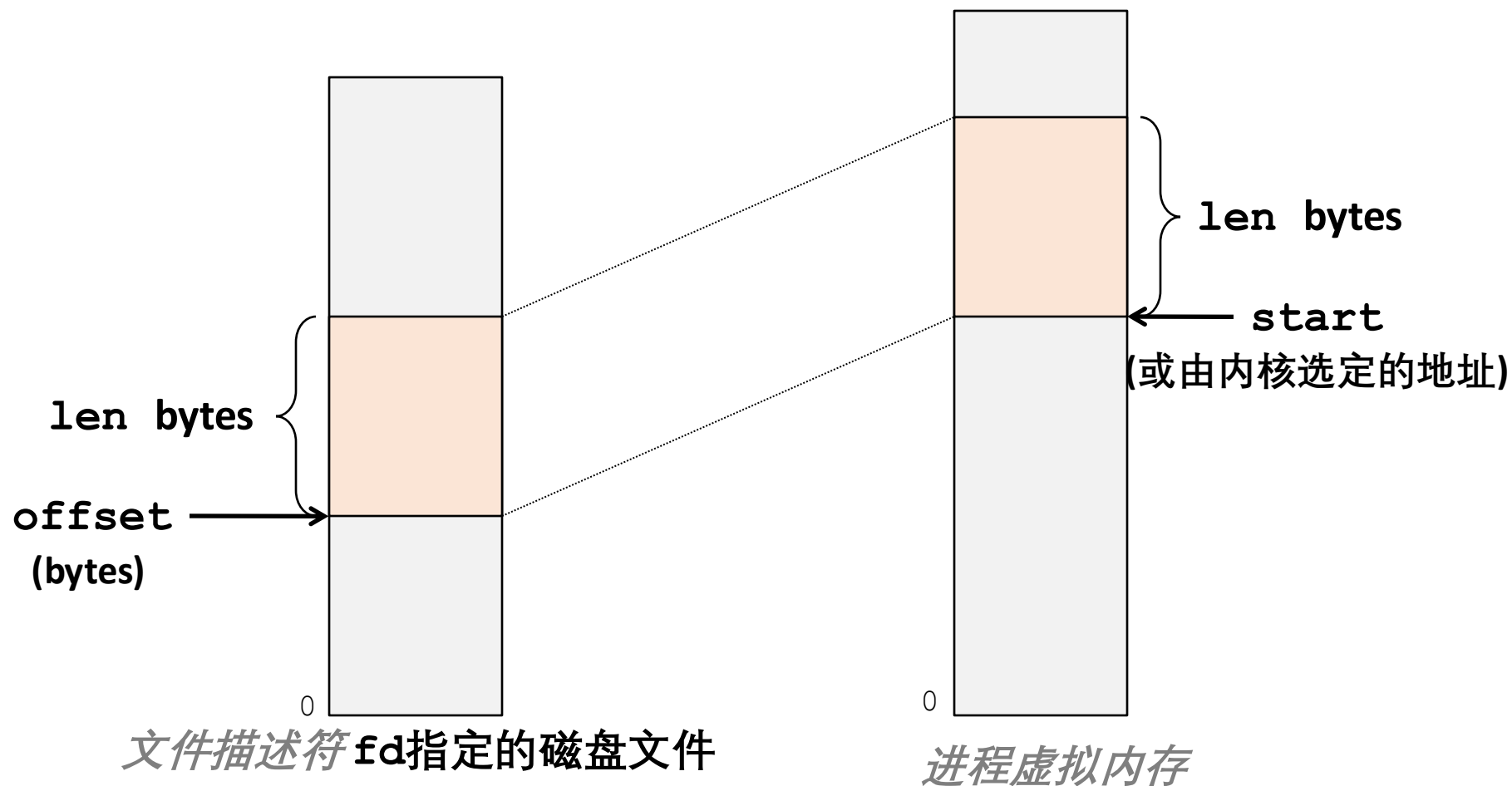
```
void *mmap(void *start, int len,  
           int prot, int flags, int fd, int offset)
```

创建新的虚拟内存区域，并将对象映射到这些区域

- 从fd指定的磁盘文件的offset处映射len个字节到一个新创建的虚拟内存区域，该区域从地址start处开始
 - **start**: 虚拟内存的起始地址，通常定义为NULL
 - **prot**: 虚拟内存区域的访问权限，PROT_READ, PROT_WRITE, ...
 - **flags**: 被映射对象的类型，MAP_ANON（匿名对象），MAP_PRIVATE（私有的写时复制对象），MAP_SHARED（共享对象），...
- 返回一个指向映射区域开始处的指针

用户级内存映射

```
void *mmap(void *start, int len,  
           int prot, int flags, int fd, int offset)
```



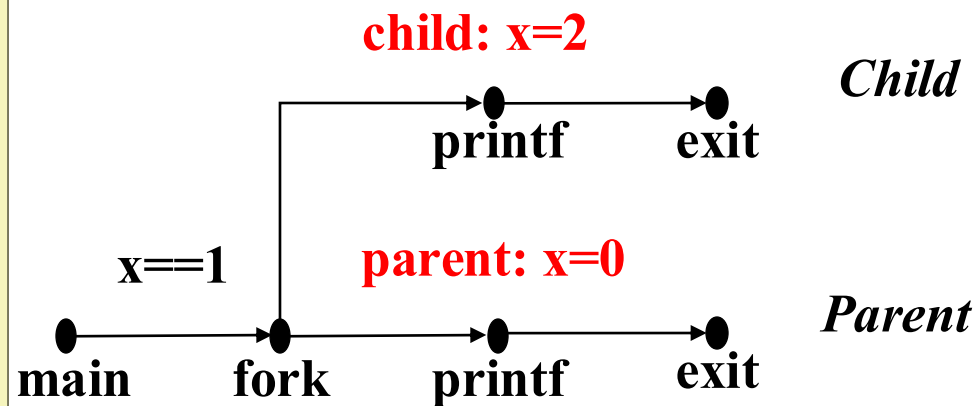
进程图

fork.c

```
int main()
{
    pid_t pid;
    int x = 1;

    pid = Fork();
    if (pid == 0) { /* Child */
        printf("child : x=%d\n", ++x);
        exit(0);
    }

    /* Parent */
    printf("parent: x=%d\n", --x);
    exit(0);
}
```

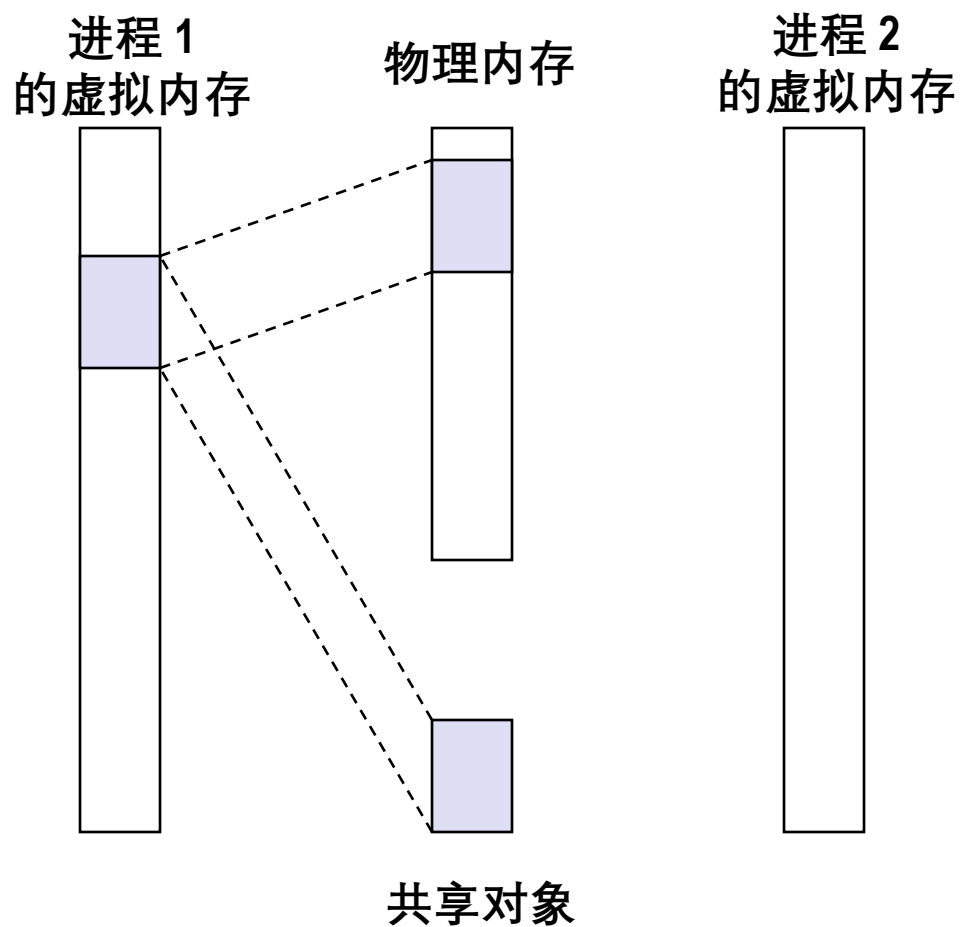


提出问题：

进程fork：

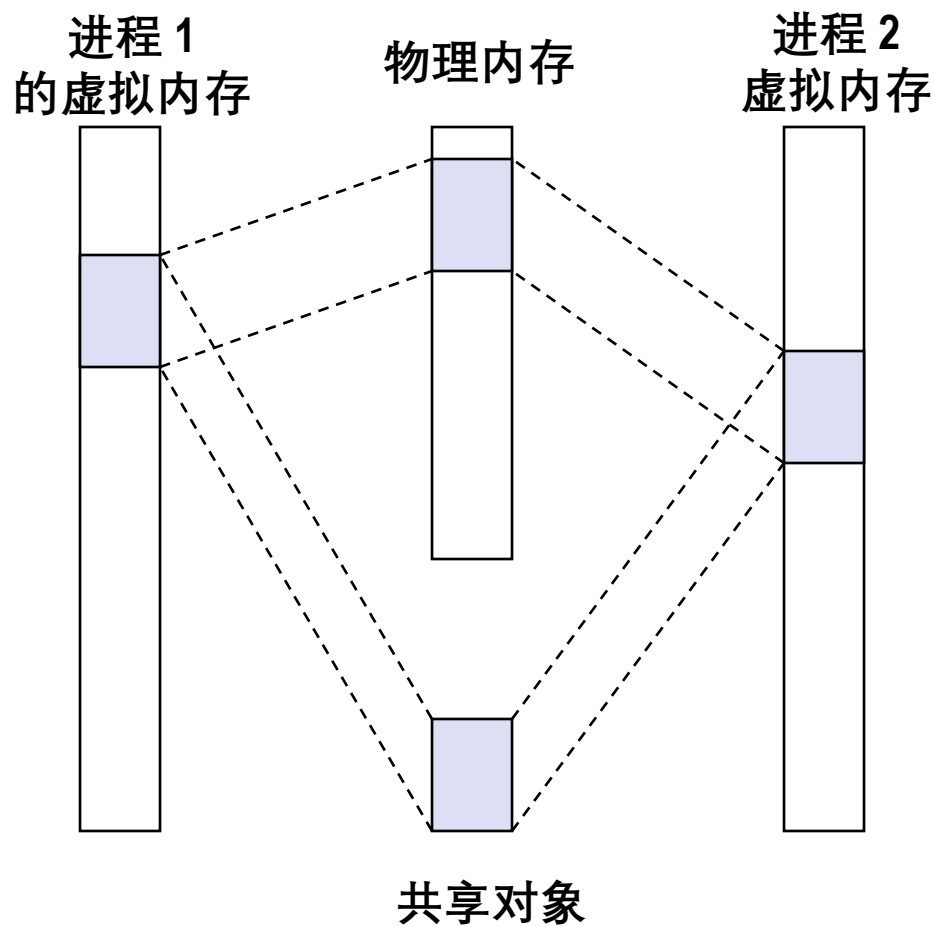
父进程、子进程具有同样的存储区域，
同时修改全局变量怎么办？

再看共享对象



- 进程 1 映射了共享对象

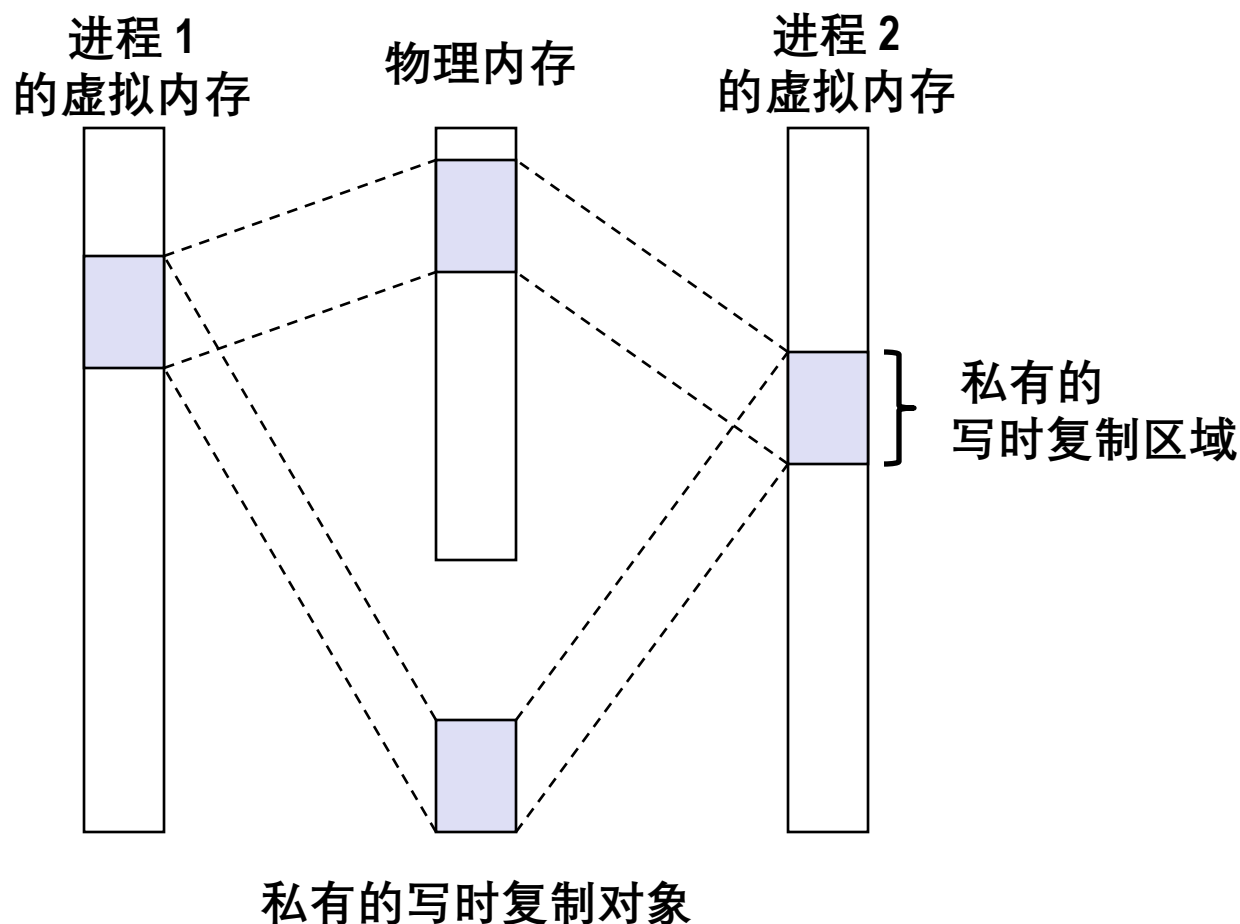
再看共享对象



- 进程 2 映射了同一个共享对象.
- 两个进程的**虚拟地址**可以是不同的

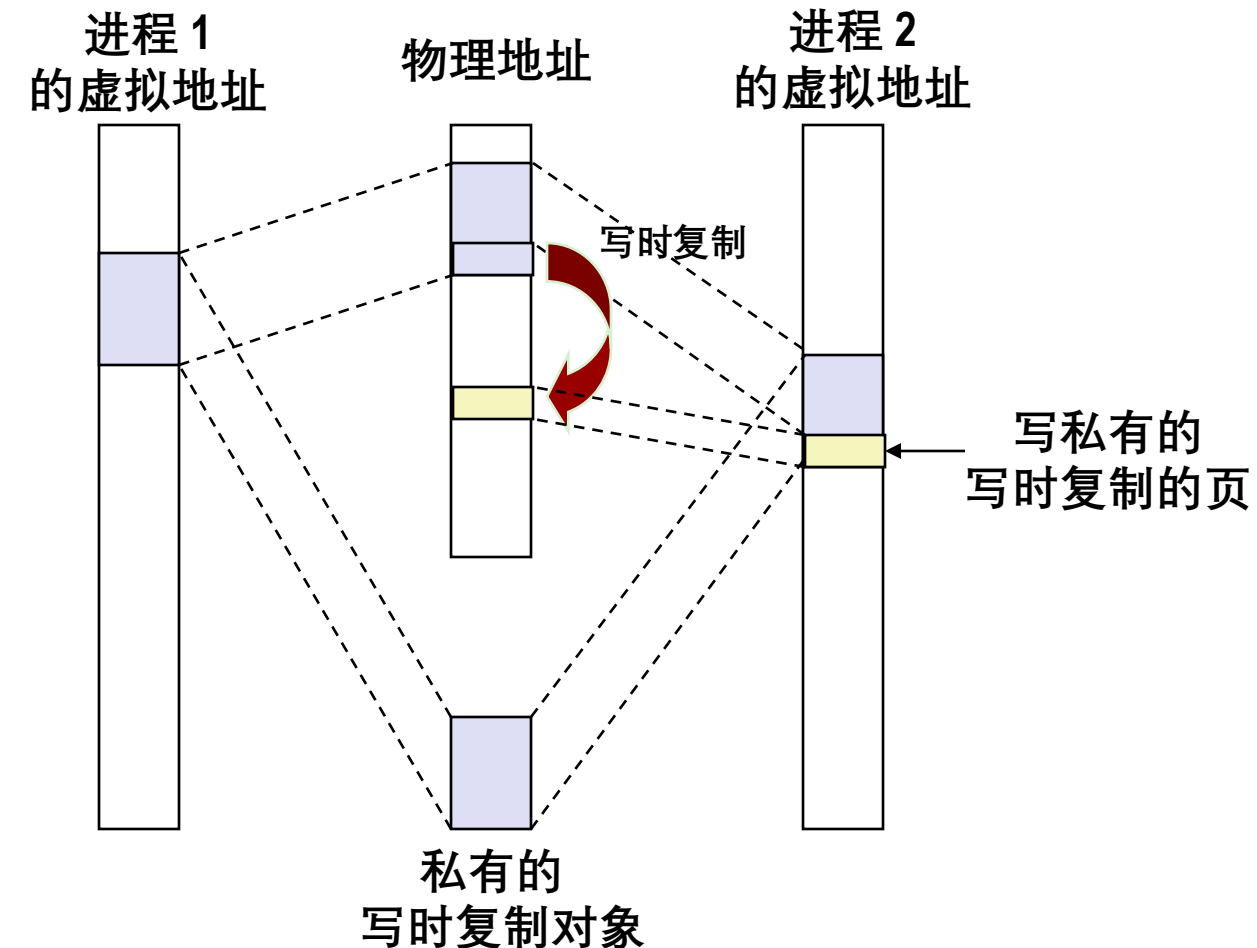
共享对象：堆栈、数据、堆区、寄存器

私有的写时复制（Copy-on-write）对象



- 两个进程都映射了私有的写时复制对象
- 区域结构被标记为私有的写时复制
- 私有区域的页表条目都被标记为只读

共享对象： 私有的写时复制（Copy-on-write）对象

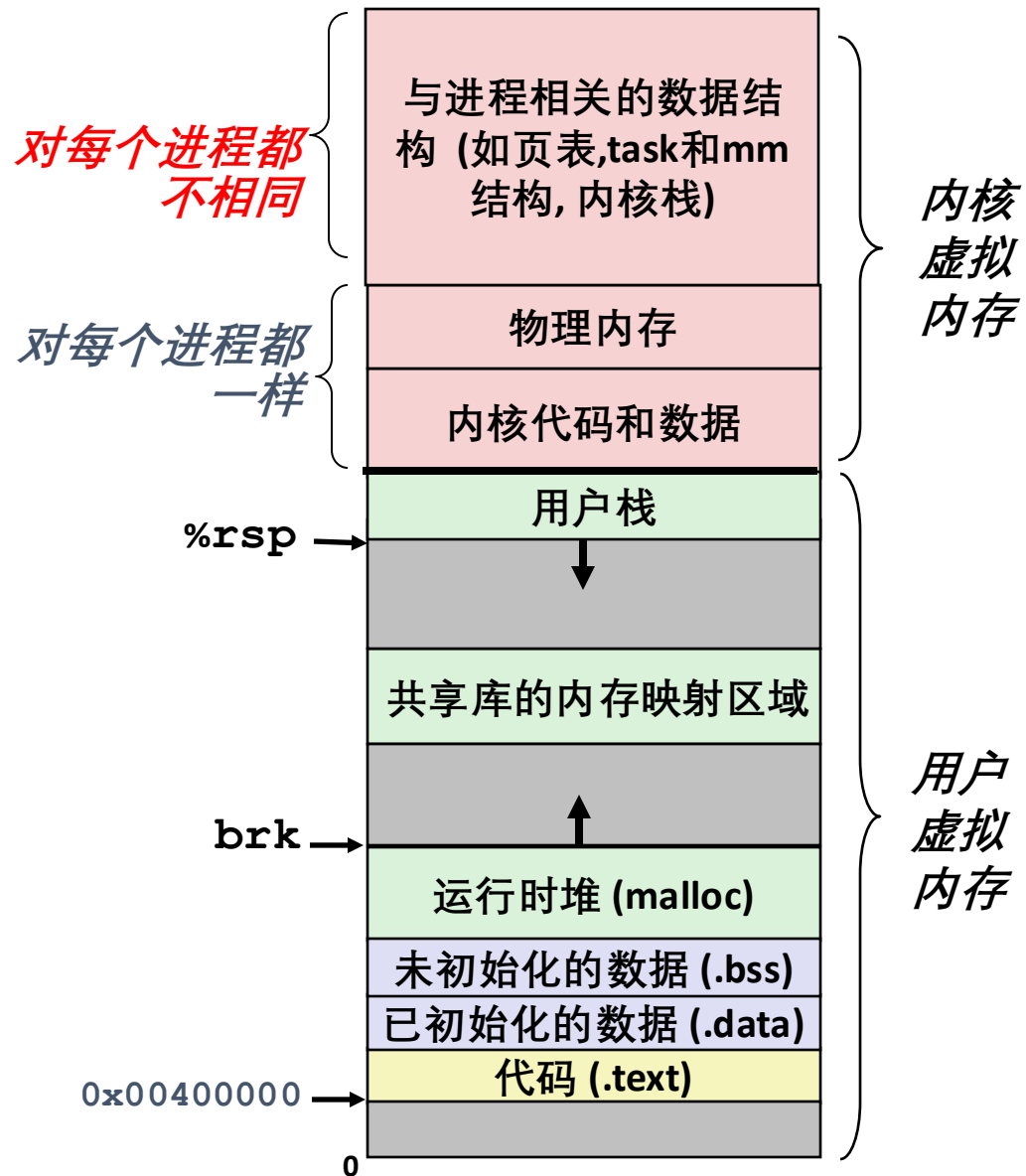


- 写私有页的指令触发保护故障
- 故障处理程序创建这个页面的一个新副本，更新PTE条目，且可写
- 故障处理程序返回时重新执行写指令
- 尽可能地延迟拷贝（创建副本）充分利用物理内存

其他技术

	COW	ROW
创建	耗时很短，创建时，占用少量存储空间	
写源卷	写性能受影响	写性能不受影响
读源卷	读性能不受影响	读性能受影响
读快照	读重定向，有轻微时延	
删除快照	瞬间完成	需要数据回拷到源卷，耗时可能很久
应用场景	读密集型系统	写密集型系统
ROW 与 COW 最大的不同就是： COW 的快照卷存放的是原始数据，而 ROW 的快照卷存放的是新数据		

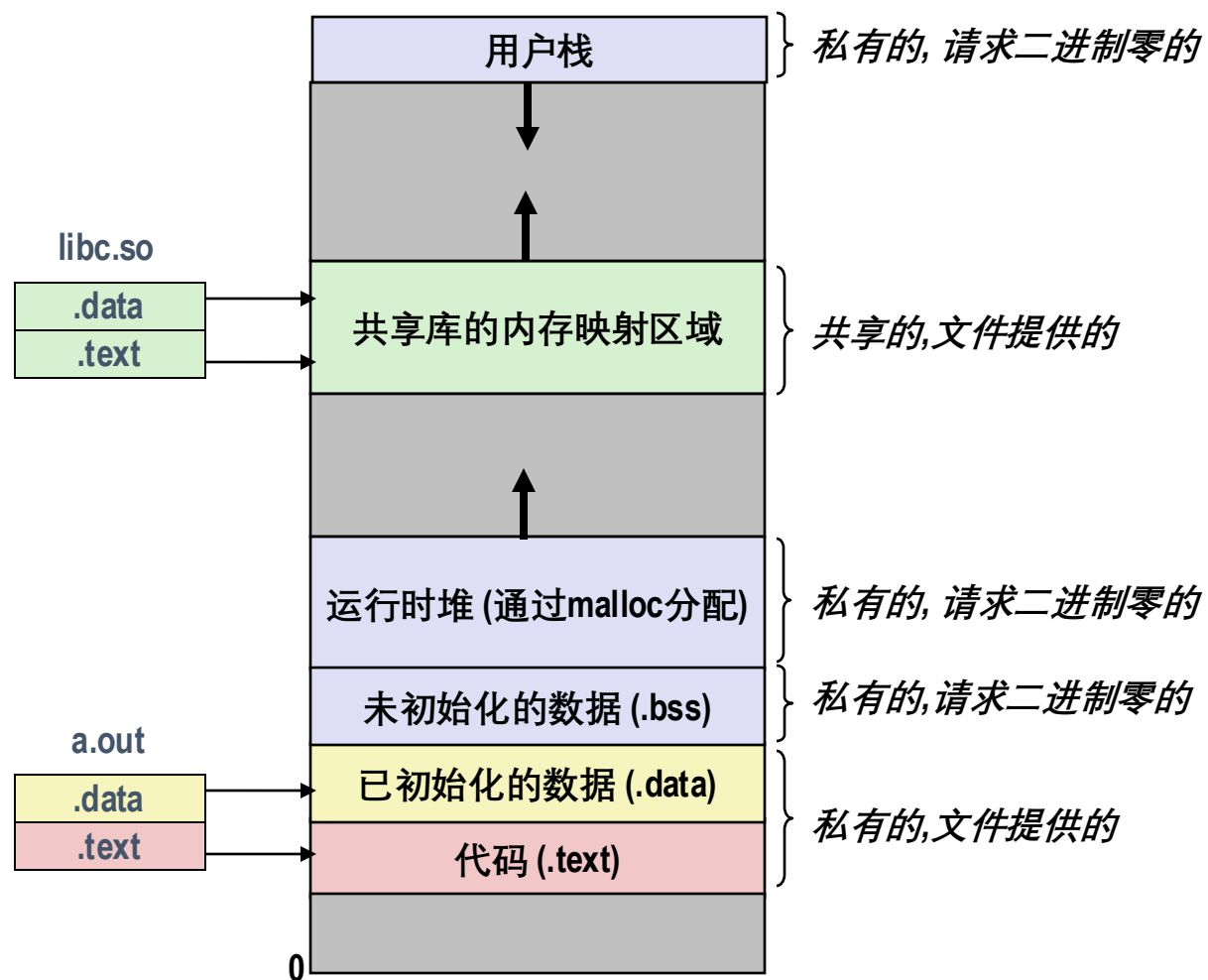
一个Linux进程的虚拟地址空间 (回顾)



再看 fork 函数

- 虚拟内存和内存映射解释了fork函数如何为每个新进程提供私有的虚拟地址空间.
- 为**新进程**创建虚拟内存
 - 创建**当前进程**的`mm_struct`, `vm_area_struct`和页表的原样副本.
 - 两个进程中的每个页面都标记为只读
 - 两个进程中的每个区域结构 (`vm_area_struct`) 都标记为私有的写时复制 (COW)
- 在新进程中返回时, 新进程拥有与调用fork进程相同的虚拟内存
- 随后的写操作通过写时复制机制创建新页面

再看 execve 函数



`execve`函数在当前进程中加载并运行新程序`a.out`的步骤:

- 删除已存在的用户区域
- 创建新的区域结构
 - 私有的、写时复制
 - 代码和初始化数据映射到`.text`和`.data`区 (目标文件提供)
 - `.bss`和栈堆映射到匿名文件, 栈堆的初始长度0
- 共享对象由动态链接映射到本进程共享区域
- 设置PC, 指向代码区域的入口点
 - Linux根据需要换入代码和数据页面

Example: 使用 mmap 函数拷贝文件

- 拷贝一个文件到 stdout （数据没有经过缓冲区）

```
#include "csapp.h"

void mmapcopy(int fd, int size)
{

    /* Ptr to memory mapped area */
    char *bufp;

    bufp = mmap(NULL, size,
                PROT_READ,
                MAP_PRIVATE,
                fd, 0);
    write(1, bufp, size);
    return;
}
```

mmapcopy.c

```
/* mmapcopy driver */
int main(int argc, char **argv)
{
    struct stat stat;
    int fd;

    /* Check for required cmd line arg */
    if (argc != 2) {
        printf("usage: %s <filename>\n",
              argv[0]);
        exit(0);
    }
    /* Copy input file to stdout */
    fd = open(argv[1], O_RDONLY, 0);
    fstat(fd, &stat);
    mmapcopy(fd, stat.st_size);
    exit(0);
}
```

mmapcopy.c

buffered I/O vs mmap

```
char buf[BUF_SIZE];  
read(fd, buf, size);  
write(1, buf, size);
```

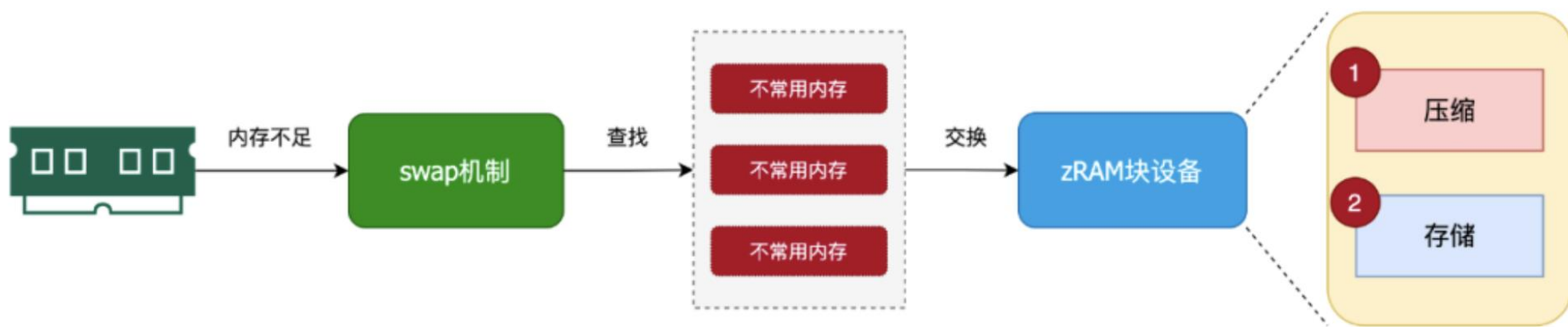
[磁盘块]
↓ ① (DMA 或 blk I/O 载入)
[Page Cache](内核页缓存)
↓ ② (copy_to_user)
[用户缓冲区 buf[] in 用户空间]
↓ ③ (copy_from_user)
[write() → 内核 socket/send buf]

```
char *bufp = mmap(...);  
write(1, bufp, size);
```

[磁盘块]
↓ ① (DMA 或 blk I/O 载入)
[Page Cache](内核页缓存)
↑ 页表映射 ⇔ bufp (用户虚拟地址)
↓ ② (copy_from_user)
[write() → 内核 socket/send buf]

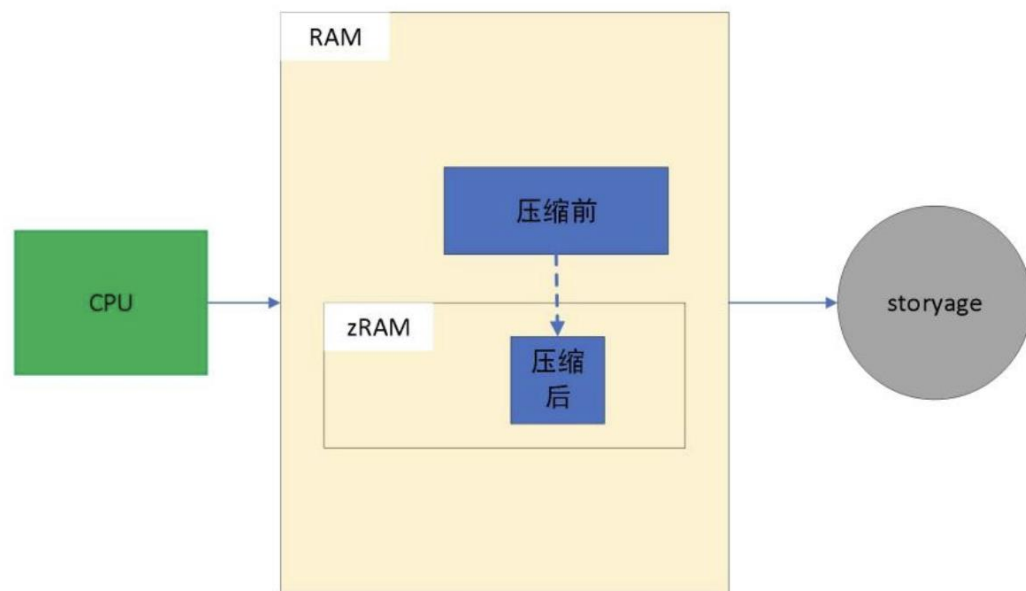
ZRAM——linux高级功能

- 物理内存空间不足时，是否可以使用其他方法减少换页？
- 数据压缩！
- 主要面向匿名页



ZRAM——linux高级功能

- 在内存空间中划分一块，模拟为块设备
- 将该模拟的块设备作为换入换出的空间
- 在该模拟设备的I/O路径上加入数据压缩/解压功能
- Android, MacOS的常用功能



ZRAM——linux高级功能

algo	page-cluster	"MiB/s"	"IOPS"	"Mean Latency (ns)"	"99% Latency (ns)"
lzo	1	9130.553415506836	1168710.932773	5951.70401	27264
lzo	2	11533.120699842773	738119.747899	9902.73897	41728
lzo	3	13018.8484358584	416603.10084	18137.20228	69120
lzo	0	6360.546916032226	1628299.941176	3998.240967	18048
zstd	1	4964.754078584961	635488.478992	11483.876873	53504
zstd	2	5908.13019465625	378120.226891	19977.785468	85504
zstd	3	6350.650210083984	203220.823529	37959.488813	150528
zstd	0	3859.24347590625	987966.134454	7030.453683	35072
lz4	1	11200.088793330078	1433611.218487	4662.844947	22144
lz4	2	15353.485367975585	982623	7192.215964	30080
lz4	3	18335.66823135547	586741.184874	12609.004058	45824
lz4	0	7744.197593880859	1982514.554622	3203.723399	9920
lz4hc	1	12071.730649291016	1545181.588235	4335.736901	20352
lz4hc	2	15731.791228991211	1006834.563025	6973.420236	29312
lz4hc	3	19495.514164259766	623856.420168	11793.367214	43264
lz4hc	0	7583.852120536133	1941466.478992	3189.297915	9408
lzo-rle	1	9641.897805606446	1234162.857143	5559.790869	25728
lzo-rle	2	11669.048803505859	746819.092437	9682.723915	41728
lzo-rle	3	13896.739553243164	444695.663866	16870.123274	64768
lzo-rle	0	6799.982996323242	1740795.689076	3711.587765	15680
842	1	2742.551544446289	351046.621849	21805.615246	107008
842	2	2930.5026999082033	187552.218487	41516.15757	193536
842	3	2974.563821231445	95185.840336	82637.91368	366592

TABLE I: Characteristics of Different Types of Memory

Category	Read Latency (ns)	Write Latency (ns)	Endurance (# of writes per bit)
SRAM	2-3	2-3	∞
DRAM	15	15	10^{18}
STT-RAM	5-30	10-100	10^{15}
PCM	50-70	150-220	10^8 - 10^{12}
Flash	25,000	200,000-500,000	10^5
HDD	3,000,000	3,000,000	∞

*Hope you
enjoyed
the
CSAPP
course!*